

Advanced Model Development for Dynamic Pricing (XGBoost & LightGBM)

This is a complete, detailed, reference-level guide for building machine learning models for dynamic pricing. It explains every step clearly, including why each line of code is written, what alternatives exist, and their drawbacks. The goal is to predict demand and simulate optimal pricing strategies that outperform rule-based systems.

Step 1: Import Libraries

Reason: We import libraries for data handling, model training, and evaluation. Pandas handles data, NumPy helps with math, sklearn provides splitting and metrics, while XGBoost and LightGBM are powerful gradient boosting algorithms.

Code:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
import xgboost as xgb
import lightgbm as lgb
```

Step 2: Load Dataset

Reason: We load the feature-engineered dataset instead of raw data because models require meaningful features to learn patterns effectively.

Code:

```
df = pd.read_csv("feature_engineered_dataset.csv")
```

Step 3: Define Target Variable

Reason: We choose 'units_sold' as the target because pricing decisions depend on demand prediction. Predicting price directly is harder and unstable.

Code:

```
y = df["units_sold"]
```

Step 4: Define Features

Reason: We remove identifiers and date because they don't directly help prediction. Including them may cause noise or leakage.

Code:

```
x = df.drop(["units_sold", "date", "store_id", "product_id"], axis=1)
```

Step 5: Train-Test Split

Reason: We split data to test generalization. Without this, the model might memorize instead of learning patterns.

Code:

```
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42 )
```

Step 6: XGBoost Model Setup

Reason: XGBoost is chosen for its ability to handle tabular data and non-linear relationships. Alternatives like linear regression fail for complex patterns.

Code:

```
xgb_model = xgb.XGBRegressor( n_estimators=100, learning_rate=0.1, max_depth=6 )
```

Step 7: Train XGBoost

Reason: Training allows the model to learn relationships between features and demand.

Code:

```
xgb_model.fit(X_train, y_train)
```

Step 8: Predict with XGBoost

Reason: Predictions estimate demand for unseen data.

Code:

```
y_pred_xgb = xgb_model.predict(X_test)
```

Step 9: Evaluate XGBoost

Reason: RMSE measures average prediction error. Lower RMSE indicates better performance.

Code:

```
rmse_xgb = np.sqrt(mean_squared_error(y_test, y_pred_xgb)) print("XGBoost RMSE:", rmse_xgb)
```

Step 10: LightGBM Model Setup

Reason: LightGBM is faster than XGBoost and handles large datasets efficiently.

Code:

```
lgb_model = lgb.LGBMRegressor( n_estimators=100, learning_rate=0.1, max_depth=6 )
```

Step 11: Train LightGBM

Reason: Train model similarly to XGBoost.

Code:

```
lgb_model.fit(X_train, y_train)
```

Step 12: Predict with LightGBM

Reason: Generate predictions for evaluation.

Code:

```
y_pred_lgb = lgb_model.predict(X_test)
```

Step 13: Evaluate LightGBM

Reason: Compare RMSE values to choose better model.

Code:

```
rmse_lgb = np.sqrt(mean_squared_error(y_test, y_pred_lgb)) print("LightGBM RMSE:", rmse_lgb)
```

Step 14: Feature Importance

Reason: This shows which features influence predictions most. Helps validate model logic.

Code:

```
xgb.plot_importance(xgb_model) import matplotlib.pyplot as plt plt.show()
```

Step 15: Backtesting Setup

Reason: Backtesting simulates how the model would perform in real scenarios using historical data.

Code:

```
df["predicted_demand"] = xgb_model.predict(X)
```

Step 16: Pricing Logic using ML

Reason: We adjust price based on predicted demand. If demand is higher than actual, increase price; otherwise decrease.

Code:

```
def optimize_price(row): if row["predicted_demand"] > row["units_sold"]: return row["price"] * 1.05 else:
return row["price"] * 0.95 df["ml_price"] = df.apply(optimize_price, axis=1)
```

Step 17: Calculate ML Revenue

Reason: Compute revenue based on ML pricing strategy.

Code:

```
df["ml_revenue"] = df["ml_price"] * df["units_sold"]
```

Step 18: Compare with Baseline

Reason: Compare ML performance against rule-based pricing.

Code:

```
df["baseline_revenue"] = df["dynamic_price"] * df["units_sold"]
```

Step 19: Revenue Lift Calculation

Reason: Measure improvement in percentage.

Code:

```
lift = (df["ml_revenue"].sum() - df["baseline_revenue"].sum()) / df["baseline_revenue"].sum() * 100
print("ML Revenue Lift (%)ate", lift)
```

Step 20: Final Interpretation

Reason: Students must explain whether ML improved results and why.

Code:

```
# Write conclusions manually
```